

Assignment No: 7

Student Details

Name: Tanushka Patil

PRN / Roll No: 123B1B036

Class: BTech

Division: A

Subject: DevOps

Title

Explore Kubernetes Container Orchestration and Deploy an Application Using Helm

Aim

To study Kubernetes architecture and understand the structure and commands of Helm for managing Kubernetes applications.

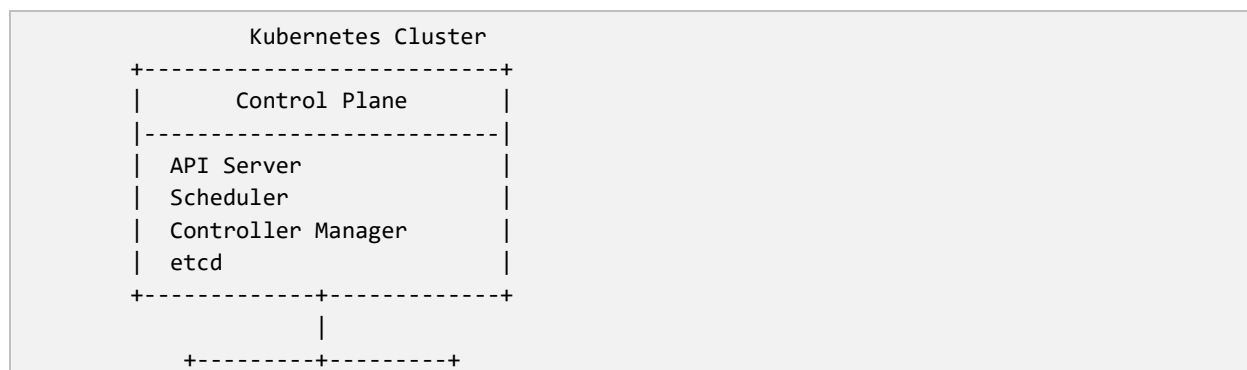
Theory

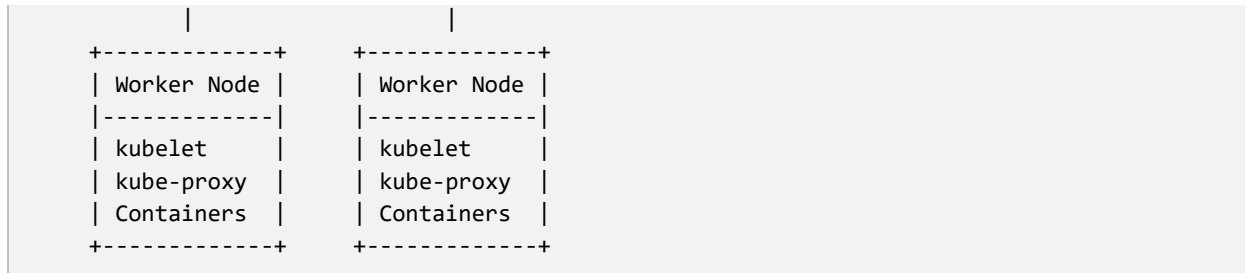
Objectives

- Understand container orchestration.
- Study Kubernetes architecture.
- Identify Kubernetes control-plane and worker-node components.
- Understand Helm and Helm charts.
- Create a basic Helm chart.
- Perform Helm install, upgrade, template and uninstall operations.

Part A: Kubernetes Architecture

Kubernetes is a container orchestration platform that automates the deployment, scaling, networking, service discovery, load balancing and recovery of containerized applications. A Kubernetes cluster consists of a control plane that makes decisions and worker nodes that run the application containers.





- **API Server:** entry point for all Kubernetes API requests (kubectl, Helm).
- **etcd:** key-value store holding the cluster configuration and state.
- **Scheduler:** assigns new Pods to suitable worker nodes.
- **Controller Manager:** runs controllers that keep the actual state equal to the desired state.
- **kubelet:** agent on each node that starts and monitors Pods and containers.
- **kube-proxy:** implements Service networking and load balancing on each node.
- **Container Runtime:** executes the containers (containerd in this cluster).

In this practical a single-node k3s cluster (a lightweight, certified Kubernetes distribution) was used. k3s runs the API server, scheduler, controller manager, kubelet and kube-proxy inside one process and uses containerd as the container runtime; CoreDNS, the metrics server and a local-path storage provisioner run as Pods in the kube-system namespace.

Part B: Helm

Helm is the package manager for Kubernetes. A Helm chart is a reusable package of templated Kubernetes manifests. Installing a chart creates a release; every upgrade creates a new revision that can be listed, inspected and rolled back.

- **Chart.yaml:** chart metadata – name, description, chart version and appVersion.
- **values.yaml:** default configuration values (replica count, image, service type) that can be overridden with --set or -f.
- **templates/:** Kubernetes YAML templates (Deployment, Service, ConfigMap, Ingress...) rendered with the values.
- **charts/:** dependent sub-charts.

Helm Commands

```

helm version           # check Helm version
helm create myapp      # create a chart
helm template myrelease ./myapp # render YAML without deploying
helm install myrelease ./myapp  # install chart as a release
helm list              # list releases
helm upgrade myrelease ./myapp  # upgrade the release
helm status myrelease   # view release status
helm history myrelease  # view revisions
helm uninstall myrelease # remove the release

```

Code / Implementation

Step 1: Kubernetes cluster and tools

A single-node k3s cluster was started in Docker, and kubectl and Helm were run from a toolbox container (alpine/k8s) connected to the cluster:

```
docker run -d --name k3s --privileged -p 127.0.0.1:30080:30080 rancher/k3s:v1.31.4-k3s1
server --disable=traefik
docker run -d --name toolbox --network container:k3s -v <assignment-folder>:/work/a7
alpine/k8s:1.31.4 sleep infinity
```

Step 2: Create the Helm chart

```
helm create myapp
```

Step 3: Customise the chart

Chart.yaml (comments removed)

```
apiVersion: v2
name: myapp
description: Sample Kubernetes application
type: application
version: 0.1.0
appVersion: "1.0"
```

values.yaml – changed values

```
replicaCount: 2

image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: "1.27-alpine"

service:
  type: NodePort
  port: 80
  # Fixed node port so the app is reachable at http://localhost:30080
  nodePort: 30080

# Text shown on the web page (served from a ConfigMap)
page:
  message: "Deployed with Helm - DevOps Assignment 7"
```

templates/configmap.yaml – web page generated from values

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "myapp.fullname" . }}-page
  labels:
    {{- include "myapp.labels" . | nindent 4 }}
```

```

data:
  index.html: |
    <html>
      <body style="font-family: Arial; text-align: center; margin-top: 60px;">
        <h1>{{ .Values.page.message }}</h1>
        <p>Release: {{ .Release.Name }} | Revision: {{ .Release.Revision }} | Replicas: {{
        .Values.replicaCount }}</p>
        <p>Chart: {{ .Chart.Name }}-{{ .Chart.Version }} | Image: {{
        .Values.image.repository }}:{{ .Values.image.tag }}</p>
      </body>
    </html>

```

templates/service.yaml was extended to set nodePort when the service type is NodePort, and templates/deployment.yaml mounts the ConfigMap as the nginx web root and adds a checksum annotation so that pods are rolled when the page changes.

Step 4: Validate, render, install, upgrade and uninstall

```

helm lint ./myapp
helm template myrelease ./myapp -s templates/service.yaml -s templates/configmap.yaml
helm install myrelease ./myapp
helm list
kubectl get pods
kubectl get deployments
kubectl get services
helm upgrade myrelease ./myapp --set replicaCount=3 --set page.message='Upgraded with Helm
- DevOps Assignment 7'
helm history myrelease
helm status myrelease
helm uninstall myrelease

```

Output

1. Kubernetes cluster information, node and kube-system components:

```

toolbox (kubectl + helm) — connected to k3s cluster
/work/a7 # kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:6443
CoreDNS is running at https://127.0.0.1:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
Metrics-server is running at https://127.0.0.1:6443/api/v1/namespaces/kube-system/services/https:metrics-server:https/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.

/work/a7 # kubectl get nodes -o wide
NAME          STATUS    ROLES          AGE      VERSION          INTERNAL-IP  EXTERNAL-IP  OS-IMAGE          KERNEL-VERSION
k3s-node      Ready     control-plane,master  2m57s    v1.31.4+k3s1     172.17.0.3   <none>       K3s v1.31.4+k3s1   6.6.87.2-micros
  oft-standard-WSL2      containerd://1.7.23-k3s2

/work/a7 # kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
coredns-ccb96694c-nw5n1             1/1     Running   0           2m51s
local-path-provisioner-5cf85fd84d-d5bxn  1/1     Running   0           2m51s
metrics-server-5985cbc9d7-gwznd      1/1     Running   0           2m51s

```

2. Creating the Helm chart and its file structure:

```
toolbox (kubectl + helm) — connected to k3s cluster
/work/a7 # helm version --short
v3.16.3+gcfd0749

/work/a7 # helm create myapp
Creating myapp

/work/a7 # find myapp -type f | sort
myapp/.helmignore
myapp/Chart.yaml
myapp/templates/NOTES.txt
myapp/templates/_helpers.tpl
myapp/templates/deployment.yaml
myapp/templates/hpa.yaml
myapp/templates/ingress.yaml
myapp/templates/service.yaml
myapp/templates/serviceaccount.yaml
myapp/templates/tests/test-connection.yaml
myapp/values.yaml
```

3. Customised Chart.yaml and values.yaml, and helm lint result:

```
toolbox (kubectl + helm) — connected to k3s cluster
/work/a7 # cat myapp/Chart.yaml | grep -v '^#' | grep -v '^$'
apiVersion: v2
name: myapp
description: Sample Kubernetes application
type: application
version: 0.1.0
appVersion: "1.0"

/work/a7 # grep -E '^replicaCount|^image:|^ repository|^ tag|^service:|^ type|^ port|^ node
Port|^page:|^ message' myapp/values.yaml
replicaCount: 2
image:
  repository: nginx
  tag: "1.27-alpine"
service:
  type: NodePort
  port: 80
  nodePort: 30080
page:
  message: "Deployed with Helm - DevOps Assignment 7"

/work/a7 # helm lint ./myapp
==> Linting ./myapp
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed
```

4. helm template – Kubernetes YAML generated without deploying:

```
toolbox (kubectl + helm) — connected to k3s cluster
/work/a7 # helm template myrelease ./myapp -s templates/service.yaml -s templates/configmap.yaml
---
# Source: myapp/templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: myrelease-myapp
  labels:
    helm.sh/chart: myapp-0.1.0
    app.kubernetes.io/name: myapp
    app.kubernetes.io/instance: myrelease
    app.kubernetes.io/version: "1.0"
    app.kubernetes.io/managed-by: Helm
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: http
      protocol: TCP
      name: http
      nodePort: 30080
  selector:
    app.kubernetes.io/name: myapp
    app.kubernetes.io/instance: myrelease
---
# Source: myapp/templates/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: myrelease-myapp-page
  labels:
    helm.sh/chart: myapp-0.1.0
    app.kubernetes.io/name: myapp
    app.kubernetes.io/instance: myrelease
    app.kubernetes.io/version: "1.0"
    app.kubernetes.io/managed-by: Helm
data:
  index.html: |
    <html>
      <body style="font-family: Arial; text-align: center; margin-top: 60px;">
        <h1>Deployed with Helm - DevOps Assignment 7</h1>
        <p>Release: myrelease | Revision: 1 | Replicas: 2</p>
        <p>Chart: myapp-0.1.0 | Image: nginx:1.27-alpine</p>
      </body>
    </html>
```

5. helm install and helm list – release myrelease deployed (revision 1):

```
toolbox (kubectl + helm) — connected to k3s cluster
/work/a7 # helm install myrelease ./myapp
NAME: myrelease
LAST DEPLOYED: Tue Sep 29 18:06:31 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myrelease-myapp)
  export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT

/work/a7 # helm list
NAME                NAMESPACE    REVISION    UPDATED                               STATUS    CHART          APP
myrelease           default       1           2026-09-29 18:06:31.213506128 +0000 UTC deployed  myapp-0.1.0    1.0
```

6. Verification with kubectl – 2 Pods, Deployment and NodePort Service, and the page served by the application:

```
toolbox (kubectl + helm) — connected to k3s cluster

/work/a7 # kubectl rollout status deployment/myrelease-myapp --timeout=120s
Waiting for deployment "myrelease-myapp" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "myrelease-myapp" rollout to finish: 1 of 2 updated replicas are available...
deployment "myrelease-myapp" successfully rolled out

/work/a7 # kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myrelease-myapp-cbfd4c7cc-twbrk     1/1     Running   0           18s
myrelease-myapp-cbfd4c7cc-w6wrn     1/1     Running   0           18s

/work/a7 # kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
myrelease-myapp 2/2      2            2           18s

/work/a7 # kubectl get services
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes    ClusterIP   10.43.0.1    <none>        443/TCP          4m25s
myrelease-myapp NodePort    10.43.163.95 <none>        80:30080/TCP     19s

/work/a7 # curl -s http://localhost:30080
<html>
  <body style="font-family: Arial; text-align: center; margin-top: 60px;">
    <h1>Deployed with Helm - DevOps Assignment 7</h1>
    <p>Release: myrelease | Revision: 1 | Replicas: 2</p>
    <p>Chart: myapp-0.1.0 | Image: nginx:1.27-alpine</p>
  </body>
</html>
```



7. helm upgrade – replicas increased to 3 and message changed, rolling update to revision 2:

```
toolbox (kubectl + helm) — connected to k3s cluster
/work/a7 # helm upgrade myrelease ./myapp --set replicaCount=3 --set page.message='Upgraded with Helm - DevOps Assignment 7'
Release "myrelease" has been upgraded. Happy Helming!
NAME: myrelease
LAST DEPLOYED: Tue Sep 29 18:07:13 2026
NAMESPACE: default
STATUS: deployed
REVISION: 2
NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myrelease-myapp)
  export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT

/work/a7 # kubectl rollout status deployment/myrelease-myapp --timeout=120s
Waiting for deployment "myrelease-myapp" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "myrelease-myapp" rollout to finish: 1 out of 3 new replicas have been updated...
...
deployment "myrelease-myapp" successfully rolled out

/work/a7 # kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myrelease-myapp-6fd9dffd5c-5z89r    1/1     Running   0           7s
myrelease-myapp-6fd9dffd5c-pz5jt    1/1     Running   0           5s
myrelease-myapp-6fd9dffd5c-vsgdc    1/1     Running   0           3s
myrelease-myapp-cbfd4c7cc-w6wrn     1/1     Terminating   0           50s
```



8. helm list, helm history and helm status after the upgrade, and the 3 running Pods:

```
toolbox (kubectl + helm) — connected to k3s cluster

/work/a7 # helm list
NAME          NAMESPACE  REVISION  UPDATED                               STATUS  CHART          APP VERSION
myrelease     default     2         2026-09-29 18:07:13.464368315 +0000 UTC deployed  myapp-0.1.0    1.0

/work/a7 # helm history myrelease
REVISION  UPDATED                     STATUS  CHART          APP VERSION  DESCRIPTION
1         Tue Sep 29 18:06:31 2026  superseded  myapp-0.1.0    1.0         Install complete
2         Tue Sep 29 18:07:13 2026  deployed   myapp-0.1.0    1.0         Upgrade complete

/work/a7 # helm status myrelease | head -8
NAME: myrelease
LAST DEPLOYED: Tue Sep 29 18:07:13 2026
NAMESPACE: default
STATUS: deployed
REVISION: 2
NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myrelease-myapp)

/work/a7 # kubectl get pods -o wide
NAME                                READY  STATUS   RESTARTS  AGE  IP            NODE        NOMINATED NODE  READINESS GATES
myrelease-myapp-6fd9dffd5c-5z89r    1/1    Running  0          49s  10.42.0.8     k3s-node    <none>          <none>
myrelease-myapp-6fd9dffd5c-pz5jt    1/1    Running  0          47s  10.42.0.9     k3s-node    <none>          <none>
myrelease-myapp-6fd9dffd5c-vsgdc    1/1    Running  0          45s  10.42.0.10    k3s-node    <none>          <none>

/work/a7 # kubectl get deployment myrelease-myapp
NAME          READY  UP-TO-DATE  AVAILABLE  AGE
myrelease-myapp  3/3    3           3          93s
```

9. helm uninstall – release and all its Kubernetes resources removed:

```
toolbox (kubectl + helm) — connected to k3s cluster

/work/a7 # helm uninstall myrelease
release "myrelease" uninstalled

/work/a7 # helm list
NAME          NAMESPACE  REVISION  UPDATED                               STATUS  CHART          APP VERSION

/work/a7 # kubectl get pods,deployments,services
NAME                                TYPE        CLUSTER-IP  EXTERNAL-IP  PORT(S)    AGE
service/kubernetes                  ClusterIP    10.43.0.1    <none>       443/TCP    5m40s
```

Conclusion

In this assignment, I learned about Kubernetes architecture – the control-plane components (API Server, etcd, Scheduler, Controller Manager) and node components (kubelet, kube-proxy, container runtime) – and about Helm as the package manager for Kubernetes. I created a Helm chart, customised Chart.yaml, values.yaml and the templates, rendered it with helm template, installed it as a release, upgraded it to 3 replicas with a new message (revision 2) through a rolling update, inspected it with helm list, status and history, and finally removed it with helm uninstall.

References

1. Kubernetes Components – <https://kubernetes.io/docs/concepts/overview/components/>
2. Helm Documentation – <https://helm.sh/docs/>
3. Helm Chart Template Guide – https://helm.sh/docs/chart_template_guide/
4. K3s Documentation – <https://docs.k3s.io/>
5. kubectl Reference – <https://kubernetes.io/docs/reference/kubectl/>

Submission Links

GitHub Repository Link:

https://github.com/tanushkapatil/DevOps/tree/main/Assignment7_K8s_Helm

Hosted / Output Link: <http://localhost:30080> (local NodePort – release uninstalled after testing)